Les interruptions du PIC16F84A

Polo Antoine

1. INTRODUCTION

Dans l'introduction au PIC16F84A, nous avons fait clignoter une LED à l'aide de la fonction bloquante `__delay_ms()`. Cette approche est simple, mais le microcontrôleur ne peut rien faire d'autre pendant qu'il attend.

Dans ce document, nous allons voir comment utiliser les interruptions pour réagir à un événement (un appui sur un bouton, par exemple) sans interrompre le programme principal. Nous détaillerons pour cela les registres `INTCON` et `OPTION_REG`.

2. LES INTERRUPTIONS

2.1. Sources d'interruption

Une interruption suspend temporairement le programme principal pour exécuter une fonction particulière, la routine d'interruption, puis le programme reprend là où il s'était arrêté. Le PIC16F84A possède plusieurs sources d'interruption :

- une interruption configurable sur la broche RB0 ;
- une interruption sur les broches RB4 à RB7, qui s'active lors de tous les changements d'état ;
- une interruption pour le débordement du Timer0 ;
- une interruption pour la fin de l'écriture dans l'EEPROM.

2.2. Registre de gestion des interruptions (INTCON)

Le registre `INTCON` contient les bits d'activation (*enable*) et les drapeaux (*flags*) des sources d'interruption.

Table 1. Bits du registre INTCON.

b7	b6	b5	b4	b3	b2	b1	b0
GIE	EEIE	T0IE	INTE	RBIE	T0IF	INTF	RBIF

- `GIE` : lorsqu'il est mis à 1, il active les interruptions de manière globale, lorsqu'il est mis à 0, elles sont toutes coupées.
- `EEIE` : lorsqu'il est mis à 1, il autorise l'interruption de fin d'écriture d'EEPROM.
- `T0IE` : lorsqu'il est mis à 1, autorise l'interruption de Timer0 lorsqu'il passe de 255 à 0.
- `INTE` : lorsqu'il est mis à 1, autorise l'interruption sur la broche RB0/INT.
- `RBIE` : lorsqu'il est mis à 1, autorise les interruptions de changement d'état des broches RB4 à RB7.
- `T0IF` : est à 1 lorsque l'interruption de Timer0est active.
- `INTF` : est à 1 lorsque l'interruption externe est active.
- `RBIF` : est à 1 lorsque l'interruption de changement d'état est active.

Note

Les drapeaux ne sont jamais remis à 0 automatiquement : c'est au programme de le faire dans la routine d'interruption, sans quoi elle serait relancée en boucle.

Pour qu'une source déclenche une interruption, il faut donc que son bit d'activation (`INTE`, `T0IE`…) et `GIE` soient tous les deux à 1.

2.3. Registre de configuration (OPTION_REG)

Ce registre de 8 bits configure le Timer0 ainsi que quelques fonctions du PORTB. Dans l'exemple de ce document, seuls `nRBPU` et `INTEDG` nous intéressent ; les autres bits servent au Timer0.

Table 2. Bits du registre OPTION_REG.

b7	b6	b5	b4	b3	b2	b1	b0
nRBPU	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0

- `nRBPU` : contrôle les résistances de pull-up du PORTB ; le pull-up est activé à 0.
- `INTEDG` : sens de l'interruption externe (RB0), 1 pour front montant et 0 pour descendant.
- `T0CS` : source d'horloge du timer, 0 pour l'horloge interne et 1 pour l'horloge externe.
- `T0SE` : front d'horloge externe (si T0CS à 1), 0 pour front montant et 1 pour descendant.
- `PSA` : assignation du prédiviseur, à 0 il est assigné à Timer0 et à 1 pour le watchdog.
- `PS2-0` : détermine la valeur du prédiviseur (2, 4, 8, 16, 32, 64, 128, 256). Il utilise une valeur binaire de 000 à 111.

3. EXEMPLE : INTERRUPTION EXTERNE

3.1. Montage

Le montage reprend le circuit de base de l'introduction (quartz, condensateurs et résistance de reset), auquel on ajoute :

- un quartz de 8 MHz entre les broches OSC1 et OSC2, chacune reliée à la masse par un condensateur de 22 pF ;
- une résistance de 10 kΩ entre la broche MCLR et le +5 V ;
- une première LED, avec sa résistance de limitation, sur la broche RA0 ;
- une seconde LED, avec sa résistance de limitation, sur la broche RB1 ;
- un bouton poussoir entre la broche RB0/INT et la masse, sans résistance externe puisque le pull-up interne du PORTB est activé.

3.2. Code

Le programme suivant permet de faire clignoter une LED à l'aide d'un delay (bloquant) et de permettre dans le même temps de gérer un bouton poussoir pour allumer ou éteindre une deuxième LED.

```
#pragma config FOSC = HS
#pragma config WDTE = OFF
#pragma config PWRTE = OFF
#pragma config CP = OFF

#include <xc.h>

#define _XTAL_FREQ 8000000

volatile uint8_t state_led2 = 0;

void __interrupt() ISR(void) {
    if (INTCONbits.INTF) {
        state_led2 = !state_led2;
        PORTBbits.RB1 = state_led2;
        INTCONbits.INTF = 0;
    }
}

void main(void) {
```

```
    TRISAbits.TRISA0 = 0;
    TRISBbits.TRISB1 = 0;
    TRISBbits.TRISB0 = 1;

    PORTA = 0x00;
    PORTB = 0x00;

    OPTION_REGbits.nRBPU = 0;
    OPTION_REGbits.INTEDG = 0;

    INTCONbits.INTF = 0;
    INTCONbits.INTE = 1;
    INTCONbits.GIE = 1;

    while (1) {
        PORTAbits.RA0 = 1;
        __delay_ms(500);
        PORTAbits.RA0 = 0;x
        __delay_ms(500);
    }
}
```

Code 1. Interruption externe sur RB0/INT.

3.3. Fonctionnement

- **Entrées/sorties** : RA0 et RB1 sont des sorties (`TRIS = 0`), RB0 est une entrée (`TRISB0 = 1`) et les deux ports sont initialisés à 0.
- **OPTION_REG** : `nRBPU = 0` active les pull-up du PORTB, le bouton relié à la masse fait donc passer RB0 de 1 à 0 lorsqu'on appuie. `INTEDG = 0` choisit le front descendant : l'interruption se déclenche à l'appui.
- **INTCON** : `INTF` est remis à 0 pour partir d'un état propre, puis `INTE = 1` et `GIE = 1` autorisent l'interruption sur RB0/INT.
- **Boucle principale** : la LED de RA0 clignote avec `__delay_ms(500)`, comme dans l'introduction.
- **Routine d'interruption** : à chaque appui, `INTF` passe à 1 et `ISR` s'exécute, même au milieu d'un `__delay_ms()`. Elle inverse `state_led2`, l'applique à RB1 puis remet `INTF` à 0. La variable est `volatile` car elle est modifiée dans l'interruption.

Note

Un bouton mécanique rebondit : un appui peut produire plusieurs fronts descendants et donc plusieurs inversions de la LED. Il faut donc placer de 100nF sur le bouton.

4. CONCLUSION

Les interruptions permettent de réagir à un événement sans interroger en permanence une broche, et sans bloquer le programme principal. Le principe est toujours le même : on autorise la source dans `INTCON` (son bit `xxIE` ainsi que `GIE`), on configure son comportement (ici le front de RB0 et le pull-up dans `OPTION_REG`), puis la routine d'interruption traite l'événement et remet le drapeau à 0.